

# Réinitialisation du mot de passe avec OTP

SICORE — Flow, API JSON, fichiers et rôles

E-mail → Envoi de l'OTP → Vérification de l'OTP → Reset token → Nouveau mot de passe

## 1. Présentation générale

OTP signifie **One-Time Password**, c'est-à-dire un code à usage unique. SICORE divise la réinitialisation en trois étapes. Le front affiche les formulaires, appelle l'API et conserve temporairement l'état en session. Le back effectue les opérations sensibles : génération et vérification du code, envoi de l'e-mail et changement du mot de passe.

## 2. Étape 1 — Demande et envoi du code

1. L'utilisateur saisit son adresse e-mail.
2. Le front valide son format et appelle **POST /api/send-otp**.
3. Le back recherche le compte et génère un code aléatoire de six chiffres avec `random_int`.
4. Tout ancien code associé à l'e-mail est supprimé.
5. Le nouvel OTP est haché, stocké pour 10 minutes puis envoyé par e-mail.
6. Le front place l'adresse dans la session `reset_email` et ouvre l'écran de vérification.

### Requête JSON

```
{
  "email": "utilisateur@example.com"
}
```

### Réponse HTTP 200

```
{
  "message": "Code de verification envoye avec succes. Verifiez votre boite email.",
  "step": "send-otp"
}
```

**En cas d'échec SMTP** : le back supprime l'OTP créé afin de ne pas conserver un code actif que l'utilisateur n'a pas reçu.

## 3. Étape 2 — Vérification de l'OTP

1. L'utilisateur saisit le code reçu.
2. Le front impose exactement six chiffres.
3. Il appelle **POST /api/verify-otp** avec le code et l'e-mail conservé en session.
4. Le back vérifie l'existence de la demande, l'expiration et la correspondance avec le hash.
5. Si le code est valide, le back génère un `reset_token` aléatoire de 64 caractères.
6. Le token est haché en base, valable 15 minutes, et sa version utilisable est conservée dans la session du front.

### Requête JSON

```
{
  "email": "utilisateur@example.com",
  "otp": "123456"
}
```

## Réponse HTTP 200

```
{
  "message": "Code OTP verifie avec succes. Utilisez le reset_token pour modifier le mot de passe.",
  "step": "verify-otp",
  "reset_token": "jeton-temporaire-de-64-caracteres"
}
```

**Pourquoi un reset token ?** Il constitue la preuve temporaire que l'OTP a été validé. Connaître uniquement l'e-mail ne suffit donc pas pour modifier le mot de passe.

## 4. Étape 3 — Nouveau mot de passe

1. L'utilisateur saisit et confirme le nouveau mot de passe.
2. Le front impose au moins huit caractères et une confirmation identique.
3. Il appelle **POST /api/reset-password-otp**.
4. Le back exige une vérification OTP préalable, un token correct et non expiré.
5. Le nouveau mot de passe est haché puis enregistré.
6. La demande OTP est supprimée et les anciens jetons de connexion sont invalidés.
7. Le front efface `reset_email` et `reset_token`, puis redirige vers la connexion.

### Requête JSON

```
{
  "email": "utilisateur@example.com",
  "reset_token": "jeton-temporaire-de-64-caracteres",
  "password": "NouveauMotDePasse@2026",
  "password_confirmation": "NouveauMotDePasse@2026"
}
```

### Réponse HTTP 200

```
{
  "message": "Mot de passe modifié avec succès. Vous pouvez maintenant vous connecter.",
  "step": "reset-password-otp"
}
```

## 5. Protections mises en place

- OTP aléatoire à six chiffres, valable 10 minutes.
- OTP et reset token stockés uniquement sous forme hachée.
- Reset token aléatoire de 64 caractères, valable 15 minutes.
- Suppression de l'ancien OTP lors d'un nouvel envoi et après utilisation.
- Mot de passe final haché et confirmation obligatoire.
- Invalidation des anciennes connexions.
- Routes du front réservées aux visiteurs avec `guest`.
- Limitation des requêtes : 6 par minute pour l'envoi et la vérification, 3 par minute pour le renvoi.

## 6. Fichiers du front et leurs rôles

| Fichier                                                           | Rôle                                                                                                                                                                       |
|-------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>src/Http/routes/web.php</code>                              | Déclare les routes des trois étapes et du renvoi de code. Applique <code>guest</code> et les limites <code>throttle</code> .                                               |
| <code>src/Http/Controllers/AuthController.php</code>              | Contrôleur principal du flow front : validation, appels HTTP au back, gestion de <code>reset_email</code> et <code>reset_token</code> en session, erreurs et redirections. |
| <code>resources/views/pages/auth/login.blade.php</code>           | Contient le lien « Mot de passe oublié ? » qui démarre le parcours.                                                                                                        |
| <code>resources/views/pages/auth/forgot-password.blade.php</code> | Premier écran : formulaire de saisie de l'e-mail.                                                                                                                          |

|                                                     |                                                                                                |
|-----------------------------------------------------|------------------------------------------------------------------------------------------------|
| resources/views/pages/auth/verify-otp.blade.php     | Deuxième écran : saisie du code à six chiffres, indication des 10 minutes et bouton de renvoi. |
| resources/views/pages/auth/reset-password.blade.php | Troisième écran : saisie et confirmation du nouveau mot de passe.                              |
| .env / API_BASE_URL                                 | Indique au front l'URL de base du back, par exemple<br>http://localhost:8000/api.              |

**Rôle global du front :** il présente le parcours et orchestre les appels. Il ne génère pas le code et ne modifie jamais directement la base de données.

## 7. Fichiers du back et leurs rôles

| Fichier                                                                   | Rôle                                                                                                                                                                                                                          |
|---------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| scoreBack/routes/api.php                                                  | Expose /send-otp, /verify-otp et /reset-password-otp.                                                                                                                                                                         |
| app/Http/Controllers/Api/AuthController.php                               | Reçoit les requêtes validées et délègue chaque action au service d'authentification. Il joue le rôle d'intermédiaire HTTP.                                                                                                    |
| app/Http/Requests/Auth/SendOtpRequest.php                                 | Valide la présence et le format de l'e-mail.                                                                                                                                                                                  |
| app/Http/Requests/Auth/VerifyOtpRequest.php                               | Valide l'e-mail et impose un OTP de six chiffres.                                                                                                                                                                             |
| app/Http/Requests/Auth/ResetPasswordOtpRequest.php                        | Valide l'e-mail, le reset token, le minimum de huit caractères et la confirmation du mot de passe.                                                                                                                            |
| app/Services/Auth/AuthService.php                                         | Contient toute la logique métier : génération, hachage, expiration et envoi de l'OTP ; vérification du code ; création du reset token ; hachage du nouveau mot de passe ; nettoyage et invalidation des anciennes connexions. |
| app/Mail/OtpMail.php                                                      | Définit le sujet du mail, sélectionne la vue e-mail et lui transmet l'OTP et l'utilisateur.                                                                                                                                   |
| resources/views/emails/otp.blade.php                                      | Modèle HTML du message reçu : nom de l'utilisateur, code et avertissement d'expiration.                                                                                                                                       |
| database/migrations/2026_08_17_162420_create_password_reset_otp_table.php | Crée la table contenant l'e-mail, l'OTP haché, le token haché, expires_at, verified_at et created_at.                                                                                                                         |
| config/mail.php et .env                                                   | Configurent le serveur SMTP utilisé par Laravel pour envoyer le mail.                                                                                                                                                         |

## 8. Enchaînement technique des fichiers

```
Vue Blade du front
  ↓ soumission
routes/web.php
  ↓
AuthController du front
  ↓ requête HTTP JSON
routes/api.php du back
  ↓
Form Request de validation
  ↓
AuthController API
  ↓
AuthService
  ├── table password_reset_otp
  ├── OtpMail + vue emails/otp
  └── modèle User pour le mot de passe
```

**Principe architectural :** les routes dirigent les requêtes, les contrôleurs coordonnent, les Form Requests valident, le service porte la logique métier, la migration définit le stockage et le Mailable gère l'e-mail.

## 9. Discours prêt pour la soutenance

« Le flow commence dans la vue forgot-password. La route web transmet le formulaire à l'AuthController du front, qui appelle l'endpoint send-otp du back. La route API utilise un Form Request pour valider l'e-mail, puis l'AuthController API délègue au AuthService. Ce service génère et hache le code, le stocke dans password\_reset\_otps et utilise OtpMail pour l'envoyer. Lors de la vérification, le même enchaînement contrôle le code et retourne un reset token. Enfin, le troisième appel vérifie ce token, hache le nouveau mot de passe et invalide les anciennes connexions. Cette séparation rend le code plus clair : les vues gèrent l'affichage, les contrôleurs coordonnent, les Form Requests valident et le service applique la logique métier. »

## 10. Questions possibles du jury

### Pourquoi la logique est-elle placée dans un service ?

Pour alléger le contrôleur, centraliser les règles métier et rendre le traitement plus facile à maintenir et à tester.

### Pourquoi valider au front et au back ?

La validation front améliore l'expérience, mais seule celle du back garantit la sécurité puisque l'API peut être appelée directement.

### Pourquoi hacher l'OTP et le token ?

Pour empêcher leur récupération en clair en cas d'accès non autorisé à la base.

### Pourquoi invalider les anciennes connexions ?

Parce qu'une réinitialisation peut faire suite à une compromission du compte.

**Améliorations possibles :** retourner un message neutre lorsqu'un e-mail n'existe pas et ajouter un nombre maximal d'échecs pour un même OTP en complément du throttle.